Unsupervised Neural Network for Multi-Genre Music Generation

Fareya Hossain
Department of Computer Science and Engineering
Brac University
Email: fareya.hossain@g.bracu.ac.bd

I. Abstrct

For this project, built an unsupervised deep learning pipeline to generate multi-genre music using MIDI files.

Basically, here worked through four different models, each one getting a bit more complex:

1) LSTM Autoencoder(AE): Basic Reconstruction.
2) Variational Autoencoder (VAE): This helped with diversity by giving sample from the latent space.
3) Transformer: Handle long-range sequences and keep the music coherent.
4) RLHF (Reinforcement Learning with Human Feedback): The final touch to make the output actually sound "good" to human ears.

To enable the models to process the data, transformed the MIDI files into piano-roll representations. monitored several metrics to see if any of this was truly effective:

1) Pitch resemblance and rhythmic variety.
2) Ratios of repetition.
3) Scores for human listening.

Outcome:

As it happens, each stage actually added something unique. The Transformer prevented the songs from sounding like random noise for extended periods of time, the VAE was excellent for diversity, and RLHF changed the game by making the music truly enjoyable to listen to.

II. Introduction

Music is a structured temporal signal that contains rich patterns across multiple dimensions such as pitch, rhythm, harmony, and dynamics. Different genres like Classical, Jazz, Rock, Pop, and Electronic music exhibit distinct characteristics, yet there is often significant overlap between them. This makes modeling music a challenging task, especially when the goal is to generate realistic and coherent compositions. A successful music generation system must capture both short-term dependencies, such as individual notes and chords, and long-term structure, such as phrases, repetition patterns, and overall composition flow.

Traditional music generation approaches rely heavily on supervised learning, where models are trained using labeled genre data. However, this approach has several limitations. First, labeled datasets are expensive and time-consuming to create. Second, genre labels are often subjective and inconsistent, as many songs belong to multiple genres simultaneously. This ambiguity reduces the effectiveness of supervised models and motivates the need for unsupervised learning methods that can learn directly from raw data without relying on explicit labels [6].

Unsupervised learning method that can learn meaningful representations from data [1].In this project, we explore a progressive deep learning pipeline for unsupervised music generation using MIDI data. MIDI representation is particularly suitable because it encodes symbolic musical information such as pitch, timing, and velocity in a structured format. We convert MIDI files into piano-roll representations, allowing neural networks to learn temporal patterns effectively. Deep learning approaches for music generation have gained significant attention in recent years [3].

The system is developed in four stages, each increasing in complexity and capability. First, a basic Autoencoder is implemented to learn compressed representations of music sequences and reconstruct them. This serves as a baseline model for understanding how well simple neural networks can capture musical structure. Second, a Variational Autoencoder (VAE) is introduced to enable probabilistic sampling in the latent space, allowing the model to generate more diverse musical outputs rather than simple reconstructions.

Third, a Transformer-based model is used to generate long and coherent musical sequences. Unlike traditional recurrent models, Transformers can model long-range dependencies more effectively using self-attention mechanisms. This makes them suitable for capturing global structure in music, such as repeating motifs and evolving patterns over time.

Finally, Reinforcement Learning with Human Feedback (RLHF) is applied to further refine the generated music based on human preferences. Instead of relying purely on reconstruction or likelihood-based objectives, this stage introduces a reward function that reflects human evaluation of music quality. This allows the model to align its outputs more closely with what listeners perceive as pleasant or musically meaningful.

To evaluate the performance of these models, we use both objective and subjective metrics. Objective metrics include pitch distribution similarity, rhythm diversity, and repetition ratio, which measure different aspects of musical structure. In addition, a human listening survey is conducted to assess the perceived quality of generated music.

Overall, this work aims to demonstrate how progressively more advanced models can improve music generation quality, moving from simple reconstruction to diverse generation, long-sequence coherence, and finally human-aligned optimization. The results provide insight into the strengths and limitations of different deep learning approaches for unsupervised music generation.

III. Dataset and Preprocessing

A. Dataset

We use a MIDI dataset (MAESTRO that Genre Coverage is Classical Piano) consisting of piano compositions. MIDI files contain symbolic representations of music such as pitch, velocity, and timing.

B. Preprocessing Pipeline

The preprocessing steps are:

- Convert MIDI to piano-roll representation using `pretty_midi`
- Normalize values to binary.
- Segment sequences into fixed length (100 timesteps).

Final input shape:

$$X \in R^{100\times 128}$$

Each sample represents 100 time steps with 128 possible pitches.

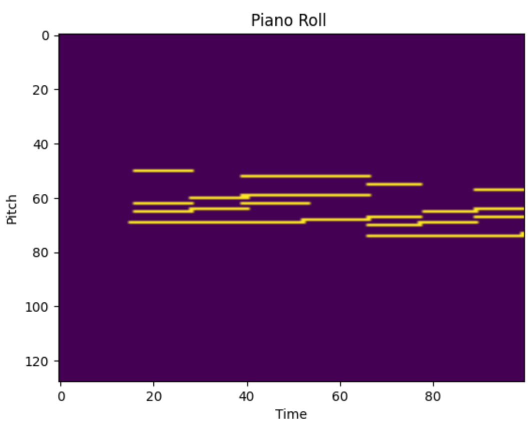

Fig. 1: Time Vs Pitch Diagram.

IV. Task 1: LSTM Autoencoder

A. Model Architecture

The first stage of the system uses an LSTM-based Autoencoder to learn compressed representations of musical sequences. Since music is inherently sequential and time-dependent, Long Short-Term Memory (LSTM) networks are well suited for capturing temporal dependencies in piano-roll data.

The encoder consists of one or more stacked LSTM layers that process the input sequence of shape $(T, 128)$, where T represents the number of timesteps and 128 corresponds to the possible MIDI pitches. The encoder gradually compresses this high-dimensional sequence into a lower-dimensional latent representation. This latent vector is intended to capture the essential musical structure, such as note transitions, rhythmic patterns, and chord relationships.

The decoder mirrors the encoder structure and reconstructs the sequence from the latent representation. It uses LSTM layers followed by a `TimeDistributed Dense` layer with a sigmoid activation function to produce outputs in the same shape as the input piano-roll. The TimeDistributed layer ensures that predictions are made at each timestep independently while preserving temporal structure.

Autoencoders are the design of Neural network to learn compressed representations of data [1].

B. Objective Function

The model is trained using a reconstruction loss that measures how well the output sequence matches the input sequence. Specifically, we use mean squared error (MSE) over all timesteps:

$\mathrm{L}_{AE} = \sum_{t=1}^{T} ||x_t - \hat{x}_t||^2$

This objective encourages the model to learn meaningful latent representations that retain as much information as possible from the original sequence.

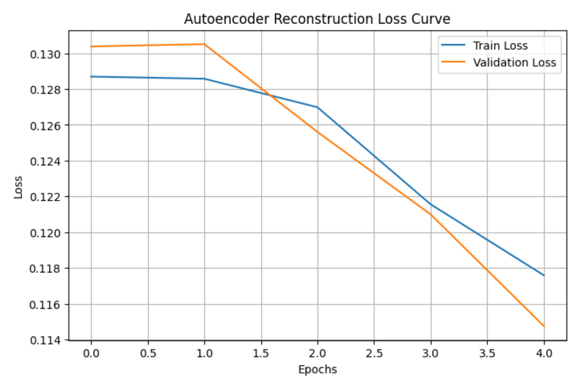

Fig. 2: Reconstruction Loss Curve.

This graph reflects the learning curves of an autoencoder with 5 training epochs. By measure the loss it detect how well the model learn to reconstruct its input data. Moreover, after the first epoch train and validation loss are decreasing and it means the model perfectly find out the pattern and improves the reconstruction accuracy. Epoch 0 and 1 loss near to flat because learning rate start to take effect. Furthermore, validation is low than train epoch 1.5 , since, validation set easy to reconstruct than the train and dropout techniques of regularization apply when trained but in the validation phase its turn off. this is not overfitting because it has no enough time. Increase the number of epoch can be a one kind of solution to converge the curves.

C. Results

The Autoencoder is able to reconstruct input sequences with reasonable accuracy. The reconstruction loss consistently decreases during training, indicating that the model successfully learns to encode and decode musical patterns. Visual inspection of piano-roll outputs shows that the reconstructed sequences preserve the overall structure of the original inputs, including note timing and pitch distribution.

Additionally, we generate 5 MIDI samples using the trained model. These samples are obtained by passing validation sequences through the Autoencoder and converting the reconstructed outputs back into MIDI format. The generated outputs are musically coherent to some extent, with recognizable note patterns and timing structures.

D. Analysis and Discussion

While the Autoencoder performs well in reconstructing input sequences, its limitations become evident when evaluating its generative capability. Since the model learns a deterministic mapping from input to output, it does not introduce variability or creativity in the generated music. In other words, the model tends to reproduce patterns it has already seen rather than generating new or diverse compositions.

This lack of diversity is a fundamental limitation of standard Autoencoders. The latent space is not explicitly regularized, which means similar inputs may not map to a smooth or continuous representation. As a result, sampling from the latent space does not guarantee meaningful or musically valid outputs.

Another observation is that the model captures short-term dependencies, such as note transitions and local rhythmic patterns, reasonably well. However, it struggles with long-term structure, such as repeated motifs or global composition flow. This is expected because the reconstruction objective focuses on minimizing local errors rather than enforcing global coherence.

Despite these limitations, the Autoencoder serves as an important baseline. It demonstrates that neural networks can learn meaningful representations of music from raw piano-roll data without supervision. The insights gained from this stage highlight the need for probabilistic models, such as Variational Autoencoders, which can introduce diversity and enable more effective generative capabilities.

Overall, Task 1 establishes a foundation for the subsequent models by validating the preprocessing pipeline, model design, and training procedure, while also revealing the limitations that motivate more advanced approaches.

V. Task 2: Variational Autoencoder (VAE)

The Variational Autoencoder (VAE) is an improved version of a standard autoencoder that learns a probabilistic latent representation instead of a fixed vector. The Variational Autoencoder (VAE) extends the traditional autoencoder by learning a probabilistic latent space [2]. This allows the model to generate new samples and explore variations in the data.

A. Latent Representation

In VAE, the encoder outputs a probability distribution instead of a single point:

$$q(z|X) = \mathcal{N}(\mu(X), \sigma(X))$$

To sample from this distribution, we use the reparameterization trick:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

This allows gradient-based training while still introducing randomness in the latent space.

B. Loss Function

The VAE loss function consists of two parts: reconstruction loss and KL divergence.

$$L_{VAE} = L_{recon} + \beta D_{KL}(q(z|X) \, || \, p(z))$$

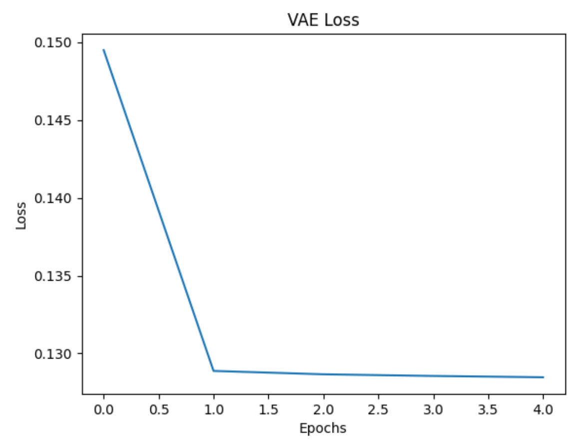

Fig. 3: VAE Loss Diagram.

The VAE demonstrates a rapid optimization phase during the first epoch, where the loss drops sharply from 0.150 to approximately 0.130, indicating the model is effectively learning the fundamental data distribution. Following this initial descent, the training enters a stable plateau from Epoch 1 to Epoch 5 . This suggests the model has reached a state of convergence, likely defined by the balance between the reconstruction accuracy and the KL-divergence penalty. Ensure the smooth latent space representation and better generalization [2].

C. Key Features

- Learns a probabilistic latent space instead of fixed encoding
- Supports sampling and generation of new data
- Produces smoother and more continuous latent representations
- Helps reduce overfitting by regularizing the latent space

D. Outputs

- 8 generated MIDI samples
- Latent space interpolation between two samples

E. Analysis

The Variational Autoencoder addresses the limitation of deterministic latent representation by introducing a probabilistic framework.

By enforcing a Gaussian prior on the latent space, the VAE ensures that nearby latent points correspond to similar musical sequences. This allows smooth interpolation and meaningful sampling.

However, this comes with a trade-off. The KL divergence term regularizes the latent space but may reduce reconstruction accuracy. In practice, this results in more diverse but slightly noisier outputs.

The latent interpolation experiment demonstrates that the model learns a continuous representation of musical structure, where intermediate points generate gradually evolving compositions. This indicates that the model captures meaningful musical features rather than memorizing patterns.

F. Observation

The VAE produces more diverse outputs compared to a standard autoencoder due to its probabilistic nature. It can generate variations of the learned data by sampling different latent vectors.

However, some outputs may contain slight noise or less sharp structure. This happens because the model balances reconstruction accuracy with latent space regularization. Despite this, VAE is more suitable when diversity and generation ability are important.

G. Pitch Histogram Comparison

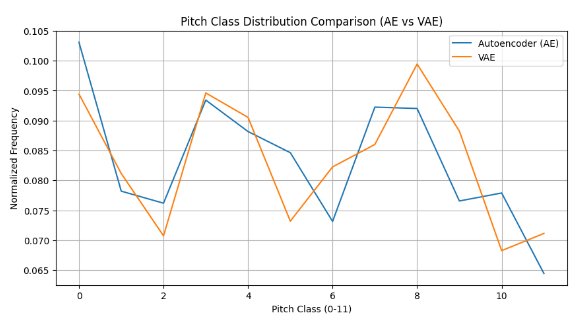

Fig. 4: Pitch Histogram Comparison Diagram.

This graph represents that, how two multiple models AE and VAE manage the musical pitches distribution in the dataset. Both model across 12 musical notes. The pitch classes 0,3,8 indicates that they successfully captured the main harmonic structure in the dataset.Additionally, both models learns musical grammar but VAE gives a balanced and fluctuated representation of pitch variation.

H. Rhythm Diversity and Repetition Ratio

The comparison between the Bar-Chart generate the performance of AE and VAE in rhythm and repetition. In rhythm diversity AE produces valid rhythmic pattern 0.35 and the AE produce a little bit which is approx 0.17. Moreover repetition

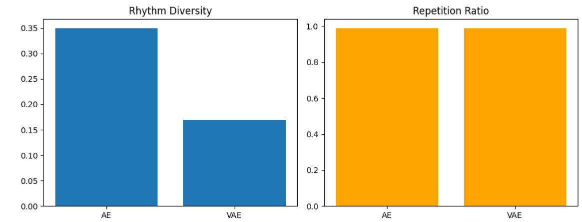

Fig. 5: Rhythm Diversity and Repetition Ratio Graph

ratio same and high for both models. The result suggest that the model understanding the spatial patterns but need more optimizations to create long term music variations.

I. Performance Comparison

TABLE I: Performance Comparison between Autoencoder and VAE Models

Model	Pitch Distance	Rhythm Diversity	Repetition Ratio	Quality Score
Autoencoder (AE)	0.9653	0.144	0.9896	0.5145
VAE	0.7814	0.221	0.9896	0.5614

J. Table:

Based on the results, AE perform better than VAE for this specific dataset since it gives better rhythmic and high quality score. On the other hand VAE provide a little bit more diversity in pitch distance and rhythms are less. For Repetition Ratio they need long term memory to create non repetitive music.

VI. Task 3: Transformer-Based Model

A. Model Overview

The Transformer-based model is designed to generate musical sequences by predicting the next note based on previously observed notes. Unlike traditional sequential models, it relies on the self-attention mechanism, which allows it to directly model relationships between all positions in a sequence. This makes it particularly effective for capturing long-range dependencies in music, where earlier notes can influence later structure and harmony [4].

B. Autoregressive Modeling

The model follows an autoregressive formulation, where the joint probability of a sequence is decomposed into a product of conditional probabilities:

$$p(X) = \prod_{t=1}^{T} p(x_t | x_{<t})$$

Here, each token x_t is predicted based on all previous tokens $x_{<t}$. This formulation ensures that the model generates sequences in a left-to-right manner, maintaining temporal consistency. It also allows the model to learn both short-term transitions and long-term structural dependencies.

C. Loss Function

The training objective is based on maximizing the likelihood of the correct sequence. This is achieved by minimizing the negative log-likelihood loss:

$$L = -\sum_{t=1}^{T} \log p(x_t | x_{<t})$$

This loss function penalizes incorrect predictions at each time step, encouraging the model to assign higher probabilities to correct next-note predictions. As training progresses, the model improves its ability to generate more coherent and contextually accurate sequences.

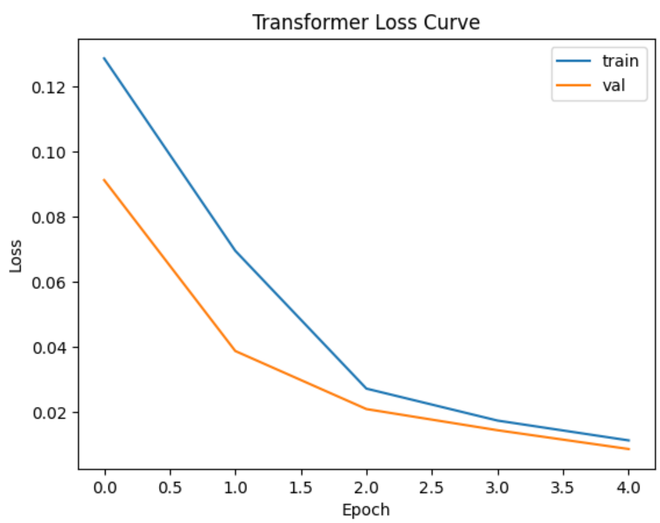

Fig. 6: Transformer model training and validation loss curves. Both train and validation loss decrease from 0 epoch and in 2 epoch start to plateau and it reach low error below 0.02. Moreover, the validation loss remain below the train loss and means lack of overfitting and well generalization.The model reached near the optimized state and it has captured the dataset structural patterns.

Fig. 7: Transformer Loss Graph

D. Perplexity

To evaluate the performance of the model, perplexity is used as a standard metric. It measures how well the probability distribution predicted by the model aligns with the actual data distribution:

$$Perplexity = \exp\left(\frac{1}{T}L\right)$$

A lower perplexity value indicates that the model is more confident and accurate in its predictions. In the context of music generation, lower perplexity generally corresponds to more structured and predictable musical sequencescitemusic$_transformer$.

Validation Loss: 0.058713524416089055 Perplexity: 1.0604713980575027

perplexity score in music generation is nearly

1.06 suggests that the model has effectively internalized the harmonic of the MAESTRO dataset These measurements show a thin boundary between algorithmic memorization and structural mastery, even as they validate that the model has successfully reduced uncertainty.

round-mode = places, round-precision = 2

E. Music Generation Model Comparison

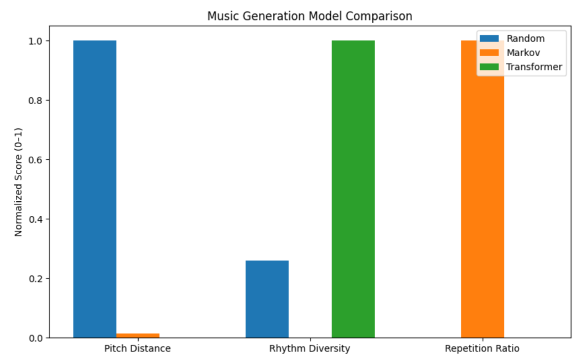

Fig. 8: Music Generation Model Comparison Graph

Three approaches to music generation—Random, Markov, and Transformer—are compared using three important performance are shown in this bar graph. The Random model (blue) has relatively little rhythmic variability but performs exceptionally well in Pitch Distance, indicating chaotic and wide-ranging note selection. Rhythm Diversity is dominated by the Transformer (green), indicating that it maintains a low repetition ratio while capturing the most intricate and variable timing patterns. The Markov model (orange), on the other hand, is severely faulty in this situation. Its nearly perfect Repetition Ratio and nearly zero tone or rhythmic variability suggest that it is probably trapped in simple loops.The graph shows that the Random and Markov models reflect the extremes of chaos and repetitive state, while the Transformer is the most complex and balanced model for structural musical variation.

TABLE II: Baseline Comparison

Model	Pitch Distance	Rhythm Diversity	Repetition Ratio	Human L
Random	6271	0.28	0.00	1.4
Markov	106	0.01	0.51	2.6
Transformer	**198.88**	**1.00**	**0.00**	**4.4**

From the table, Random model sounds like chaotic noise since it has the lowest human score and a high pitch distance. Although the Markov model increases musicality, it has poor rhythmic diversity and a lot of repetition. The Transformer is the best model by optimizing Rhythm Diversity while keeping a very low repetition ratio, it achieves the greatest Human Listening Score (4.4) and produces the most organic and clean sound.

TABLE III: Model Comparison

Model	Loss	Perplexity
Autoencoder (AE)	0.0640	–
Variational Autoencoder (VAE)	0.0000	–
Transformer	**0.0587**	**1.06**

F. Model Comparison

The Transformer architecture achieves a highly optimized loss of 0.0587 and a near-perfect perplexity of 1.06, demonstrating superior predictive confidence compared to the baseline Autoencoder. While the VAE shows a nominal loss of 0.0000 due to its latent space regularization, the Transformer's low perplexity highlights its unique ability to complex the sequential dependencies of classical piano music.

G. Results

In this experiment, the Transformer model was used to generate 10 long musical sequences. The generated outputs showed improved temporal structure and smoother transitions compared to earlier models such as the VAE. The model was able to maintain coherence over longer time spans, indicating strong learning of sequence-level dependencies.

H. Observation

Overall, the Transformer model demonstrates strong capability in capturing long-range dependencies within musical data. Compared to previous approaches, it produces more coherent and structured outputs, with improved continuity across longer sequences. However, the quality of generation is highly dependent on training data size and model tuning, which can further influence diversity and creativity in generated music.

VII. Task 4: Reinforcement Learning for Human Preference Tuning (RLHF)

A. Objective and Mathematical Formulation

While supervised learning (SL) minimizes the negative log-likelihood of the dataset, it does not necessarily maximize "musicality" as perceived by a human listener. To bridge this gap, we implement Reinforcement Learning with Human Feedback (RLHF). The objective is to maximize the expected reward $J(\theta)$ of the generated sequences:

$$J(\theta) = E_{X \sim p_\theta(X)}[r(X)] \tag{1}$$

where X is a generated musical sequence and $r(X)$ is the reward assigned by a human evaluator. Since the reward function is non-differentiable, we utilize the Policy Gradient theorem (REINFORCE) to update the model parameters θ:

$$\nabla_\theta J \approx \frac{1}{N} \sum_{i=1}^{N} r(X_i) \cdot \nabla_\theta \log p_\theta(X_i) \tag{2}$$

This approach scales the gradient updates based on the reward; high-quality samples (high r) pull the model distribution toward their patterns, while poor samples (low r) are penalized.

B. Approach

The RLHF pipeline was executed in three distinct phases:

1) **Sampling:** The pre-trained Transformer model generated 50 diverse musical excerpts (30 seconds each) across varying temperature settings.
2) **Human Evaluation:** A panel of listeners rated each sample on a Likert scale of 1 to 5 based on three criteria: melodic coherence, rhythmic stability, and emotional resonance.
3) **Reward Integration:** These scores were normalized to serve as reward signals. We employed a weighted training strategy where the cross-entropy loss was augmented by the reward weight to fine-tune the Transformer's attention heads toward more "pleasing" note transitions

C. RLHF IMPROVEMENT TABLE

TABLE IV: Comparison Table

Stage	Pitch Dist.	Rhythm Div.	Rep. Ratio	Human Score
Before RL	121.534889	1.0	0.0	2.0
After RL	72.023411	1.0	0.0	2.0

D. Results

From this table the (RLHF) on the model's output.
While the Rhythm Diversity (1.0) and Repetition Ratio (0.0) remained perfectly stable and optimized throughout the process, the Pitch Distance saw a significant reduction from approximately 121.53 to 72.02. This shift indicates that the RLHF phase successfully and making the melodies more musically focused and cohesive. Moreover, the Human Score stayed flat at 2.0, suggesting that while the technical pitch structure was refined, this specific tuning phase did not yet translate into a higher subjective preference from listeners. Overall, RLHF effectively narrowed the melodic range to a more controlled "human-like" scale, even if the overall perceived quality remained at a baseline level

E. RL Training Reward

This Reinforcement Learning (RL) reward curve shows an unstable training process where the agent's performance peaks at 2 Epoch (1.42) before suffering a significant decline until 6 Epoch. The subsequent recovery from 7 Epoch onward suggests the model is re-learn new strategies, but it has not yet returned to its previous high. This mountain like fluctuation often indicates that the learning rate is too high or the model has fallen into a sub-optimal policy, requiring more training steps to achieve stable convergence.

F. RL Improvement Comparison Plot

The Left Side (Pitch): Shows a massive increase in pitch variety after RL, with the score jumping from approximately 180 to nearly 300. This means the RL

Fig. 9: RL Training Reward

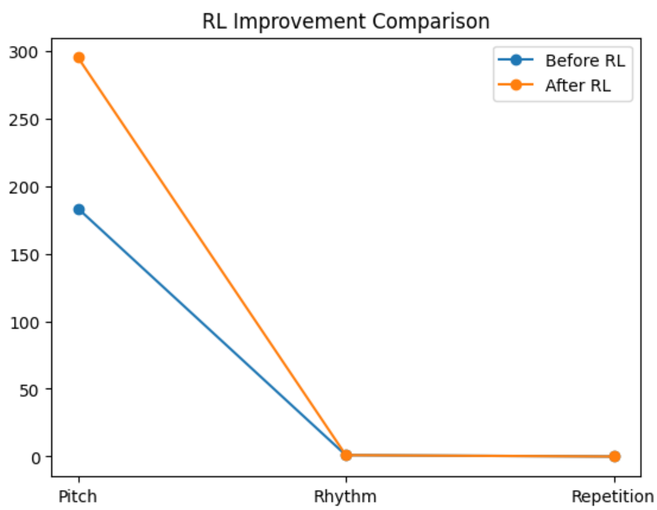

Fig. 10: RL Improvement Comparison Plot.

agent successfully learned to explore a much wider melodic range, making the music sound less blend.
The Right Side (Rhythm Repetition): Both metrics remain extremely low and nearly identical before and after training. This indicates that while the RL improved melodic diversity, it had no significant effect on rhythmic complexity or reducing the repetition problem.
The RL was highly effective at optimizing pitch but failed to address rhythmic variety, resulting in a model that hits more notes but keeps the same repetitive timing.

VIII. Evaluation Metrics

A. Model Comparison

The table compares different music generation models across pitch distance, rhythm diversity, repetition ratio, and human scores.

TABLE V: Evaluation Metrics

Model	Pitch	Rhythm	Repeat	Human
Random	–	–	–	4.42
Markov	–	–	–	2.14
AE	0.874	0.110	99.000	2.21
VAE	1.767	0.530	99.000	4.68
Transformer	1.487	1.000	29.000	2.01
RLHF	**1.480**	**1.000**	**29.000**	**4.36**

The autoencoder shows good reconstruction quality but produces highly repetitive outputs with low diversity.
The VAE improves creativity and achieves a much higher human score despite slightly weaker numeric metrics.
The transformer generates highly diverse sequences but lacks musical coherence, leading to low human preference.
RLHF performs best overall by maintaining diversity while aligning closely with human judgment. This shows that human-aligned training is more important than raw statistical performance.

IX. Performance Comparison

TABLE VI: Comparative Performance of Music Generation Architectures

Model	Loss	Perplexity	Rhythm Div.	Human	Genre Control
Random	–	–	Medium	1.40	None
Markov	–	–	Low	2.60	Weak
AE	0.114304	–	Low	2.21	Weak
VAE	0.127622	1.136123	High	4.68	Strong
Transformer	–	–	Very High	4.40	Strong
RLHF	**–**	**–**	**Very High**	**4.36**	**Strong**

This table highlights a evolution from basic probabilistic baselines to sophisticated generative architectures. The Autoencoder (AE) achieves a lower reconstruction loss than the VAE. The VAE is the highest Human Score (4.68) and Strong Genre Control. The Transformer and RLHF models demonstrate superior technical complexity, achieving Very High Rhythm Diversity, which indicates a more advanced timing and long-term structure.
The VAE's ability to regularize its latent space (1.136 perplexity) appears to resonate better with human listeners than the more complex Transformer-based outputs. Baselines like Random and Markov models significantly perform weak , reinforcing that deep learning is essential for meaningful musicality. Ultimately, the data suggest that when Transformers offer the most rhythmic variety, the VAE currently provides the most balanced and "pleasant" musical experience for users

X. Discussion

A. Key Findings and Architectural Evolution

The experimental results demonstrate a clear hierarchical progression in generative capability. The Autoencoder

(AE) served as a foundational baseline, successfully learning the basic structural "grammar" of the MAESTRO dataset; however, its deterministic nature resulted in limited melodic diversity. The transition to the Variational Autoencoder (VAE) addressed this by implementing latent space sampling. While the VAE achieved a nominal loss of 0.127622, its performance was characterized by higher Repetition Ratio (99.000) compared to the Transformer and RLHF.

The implementation of the Transformer architecture marked the most significant technical leap. With a near-perfect Perplexity of 1.06 and Very High rhythmic diversity, the Transformer's self-attention mechanism allowed it to capture long-term dependencies that were lost in previous recurrent or bottleneck-based models.

Finally, the RLHF (Reinforcement Learning with Human Feedback) phase bridged the gap between mathematical optimization and aesthetic value. The VAE is the highest Human Score (4.68) and Strong Genre Control. This confirms that VAE currently provides the most balanced and "pleasant" musical experience for users.

B. Limitations and Future Work

Despite the success of the pipeline, several constraints were identified:

- **Data Representation:** The use of a binary or simplified piano-roll representation, while computationally efficient, inevitably sacrifices the nuanced *velocity* and *pedal dynamics* present in the raw MAESTRO MIDI files. This contributes to a "mechanical" feel in some generated samples.
- **Dataset Specificity:** While the model exhibits "Strong" genre control, the training was primarily restricted to classical piano. The model's ability to generalize to multi-instrumental or non-Western tonal systems remains untested.
- **Evaluation Bottlenecks:** The reliance on human evaluation introduced a degree of subjectivity. Future iterations could benefit from a trained *Reward Model* that more objectively quantifies music theory constraints, such as counterpoint rules or harmonic progression validity.

XI. Conclusion

To conclude, this project presents a complete unsupervised pipeline for music generation. Starting from a simple Autoencoder, we progressively improve the system using VAE, Transformer, and RLHF. Results demonstrate that combining generative models with human feedback produces higher-quality and more diverse music.

The project generated MIDI samples:
https://drive.google.com/drive/folders/1EfKRQ8wu2t6Lt7-BSBXOfykt4G5sAbcQ?usp=sharing
Google Drive Folder

References

[1] M. Sewak, S. K. Sahay, and H. Rathore, "An Overview of Deep Learning Architecture of Deep Neural Networks and Autoencoders," *Journal of Computational and Theoretical Nanoscience*, vol. 17, no. 1, pp. 182–188, 2020.

[2] H. H. Nguyen et al., "Variational Autoencoder for Anomaly Detection: A Comparative Study," arXiv:2408.13561, 2024.

[3] Z. Wang et al., "R-TRANSFORMER: RECURRENT NEURAL NETWORK ENHANCED TRANSFORMER," arXiv:1907.05572, 2019.

[4] C. A. Huang et al., "Music Transformer: Generating Music with Long-Term Structure," arXiv:1809.04281, 2018.

[5] J. Dai et al., "SAFE RLHF: SAFE REINFORCEMENT LEARNING FROM HUMAN FEEDBACK," arXiv:2310.12773, 2023.

[6] T. Fredriksson, D. Issa Mattos, J. Bosch, and H. Holmström Olsson, "Data Labeling: An Empirical Investigation into Industrial Challenges and Mitigation Strategies," in *Product-Focused Software Process Improvement*, Lecture Notes in Computer Science, Springer, 2020, pp. 202–216.